GIT & GITHUB BOOTCAMP

Lesson 0

The Version Control Era

> *Welcome to the Git & GitHub Bootcamp. Before we touch a single command, we need to understand the problem that Git was created to solve. This lesson sets the stage for everything that follows.*

What You'll Learn

By the end of this lesson, students will be able to:

1. Describe the problems developers faced before version control existed.
2. Define what a Version Control System (VCS) is and why it matters.
3. Distinguish between centralized and distributed version control.
4. Explain why Git became the dominant tool in modern software development.

Life Before Version Control

Imagine you are writing an important project — a website, an app, or even a school report. You save it as project.txt. Then you make changes and you're afraid of losing the old version, so you save a copy: project_final.txt. Then another: project_final_v2.txt. Then project_final_REALLY_final.txt.

This is how most people manage files. It works for a single person on a small task, but it falls apart quickly when a project grows or when more than one person is involved.

The Problems Developers Faced

Before version control, software teams struggled with several painful problems:

1. **No reliable history —** if you broke something, there was no easy way to go back to a working version.
   - The only "backup" was a folder full of confusingly named copies.
2. **Overwriting each other's work —** when two people edited the same file, one person's changes would silently erase the other's.
   - Teams emailed ZIP files back and forth and merged changes by hand.
3. **No record of "who changed what and why" —** when a bug appeared, nobody knew which change caused it or who to ask.
4. **Fear of experimenting —** trying a new idea was risky, because there was no safe way to undo it.

> 5. *The core problem: there was no single source of truth and no time machine for code.*

The Folder-of-Copies Example

What is Version Control?

A Version Control System solves all of these problems by acting like a smart, automatic time machine for your project.

Definition

Version Control System (VCS):

> *A tool that records changes to files over time, so you can recall specific versions later, see who changed what, and work together without overwriting each other's work.*

Instead of saving messy copies, a VCS keeps one clean project and remembers every version behind the scenes.

What a VCS Gives You

Capability	What It Means For You
History	Every saved version is remembered and can be restored.
Collaboration	Many people can safely work on the same project.
Accountability	You can see who made each change and why.
Safe experimentation	Try ideas freely, knowing you can always go back.
Backup	Your work lives in more than one place.

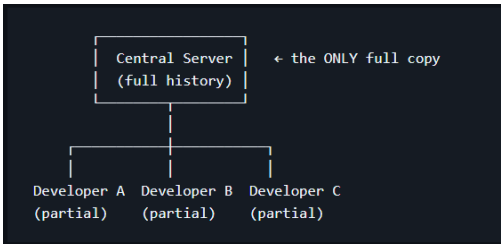
Think of a VCS as the "undo" button for an entire project — one that never forgets and works across a whole team.

Centralized vs Distributed Version Control

Not all version control systems work the same way. There are two main designs, and understanding the difference explains why Git is special.

Centralized Version Control (CVCS)

In a centralized system, there is one central server that holds the project and its full history. Developers connect to that server to get the latest files and to save their changes.



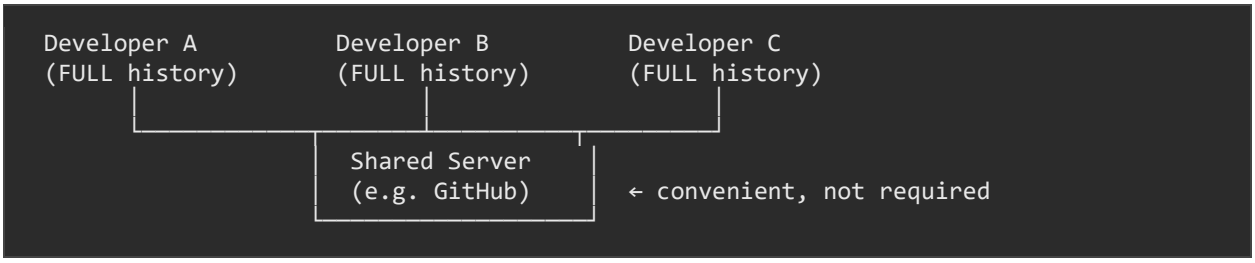
Examples include older tools like CVS and Subversion (SVN).

The big weakness:

- If the central server goes down, nobody can save work or view history.
- If the server's data is lost and there is no backup, the entire history can disappear.
- You usually need an internet connection to do almost anything.

Distributed Version Control (DVCS)

In a distributed system, every developer has a full copy of the project and its entire history on their own computer.



Git is the most famous distributed version control system.

Why this is powerful:

- You can work offline — commit, view history, and create versions without internet.
- If the shared server is lost, any developer's copy can restore everything.
- It's fast, because most actions happen on your own machine.

Side-by-Side Comparison

Aspect	Centralized (SVN)	Distributed (Git)
Where is history?	Only on the central server	On every developer's machine
Works offline?	Mostly no	Yes
Speed	Slower (network needed)	Fast (local actions)
Risk if server dies	History can be lost	Easily restored from any copy
Branching	Heavy and slow	Lightweight and fast

Why Git Became Dominant

Many version control tools have existed, but today Git is the global standard. There are clear reasons why.

1. **It's distributed and fast —** almost everything happens locally, so it's quick and works offline.
2. **It's free and open source —** anyone can use it, anywhere, at no cost.
3. **Branching is easy —** developers can create separate lines of work in seconds.
4. **It scales from tiny to huge projects —** the same tool works for a solo student project and for the Linux kernel with thousands of contributors.

5. **GitHub supercharged it —** platforms like GitHub made sharing, collaborating, and showcasing code easy, turning Git into the backbone of open source and modern teamwork.

> *A bit of history: Git was created in 2005 by Linus Torvalds, the creator of Linux, to manage the Linux kernel's massive codebase. It was built to be fast, reliable, and distributed — and those same qualities made it win.*

Common Misunderstanding

> ⚠ **"Git and GitHub are the same thing."**
>
> They are not. Git is the tool; GitHub is a website that hosts Git projects. You can use Git with no GitHub account at all. We'll clear this up completely in the next lesson.

Summary

- Before version control, developers relied on messy file copies, lost work, and had no reliable history.
- A Version Control System (VCS) records changes over time, enabling history, collaboration, and safe experimentation.
- Centralized systems keep history on one server (a single point of failure); distributed systems give every developer a full copy.
- Git is a distributed VCS — fast, offline-friendly, and resilient.
- Git became dominant because it's distributed, free, fast, great at branching, and supported by platforms like GitHub.